

Random maze generation by recursive division, and shortest-path search to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 12, 2026

Abstract

A rectangular area of $W \times H$ cells is laid out as a maze by recursive division: a full wall across the longest dimension, exactly one door in it, and the same rule inside each of the two halves. This report states that as a graph problem, shows that the requirements force the maze to be a spanning tree of the grid graph, and derives a generation algorithm together with the data structure it runs on and a proof that what it returns is that spanning tree. Giving every door unit weight makes the length of a route the number of doors crossed, which is what the shortest-path search of Section 4 minimises and what the Manhattan and Euclidean distances are compared against as heuristic guidance. A Python implementation checks the claims on generated instances, and the closing section gives the size of the structure and the cost of both algorithms.

Contents

1	Problem formulation	3
1.1	Statement	3
1.2	From a drawing of walls to a graph	3
1.3	What the specification forces E to be	4
1.4	Constraints on the exit	5
1.5	The two problems, restated on the graph	5
1.6	Conventions	5
1.7	Scope of the deliverable	5
2	Generating the maze into a graph	6
2.1	What has to be stored	6
2.2	The node record	6
2.3	Regions, named by two corners	7
2.4	The recursion stops at a corridor	7
2.5	One division contributes exactly one door	7
2.6	The region tree	8
2.7	One division step	8
2.8	The algorithm	10
2.9	Why the result is a spanning tree	10
2.10	The graph that comes out	11
3	Encoding the distance between adjacent cells as 1	11
4	Shortest path to the exit, with distance as heuristic guidance	12
5	Testing with a Python implementation	12
6	Complexity and analysis of the results	12

1 Problem formulation

1.1 Statement

A rectangular area of $W \times H$ unit cells is to be laid out as a maze.

Three things are fixed. The layout is drawn **at random**, so W and H do not determine it. The area is **enclosed** and has a single **exit**. It is entered at an arbitrary cell.

Two problems follow. Both are settled on paper first, then analysed for cost.

1. Given the width and the height of the initial area, generate a maze.
2. Find the path from any position in the maze to the exit. The specification states it is unique, hence shortest.

The construction is prescribed.

Divide the allocated space with a full **wall** across its longest dimension. Leave exactly one **door** in that wall. Apply the same rule to each of the two spaces this creates.

Figure 1 is the entire rule. Everything below follows from it.

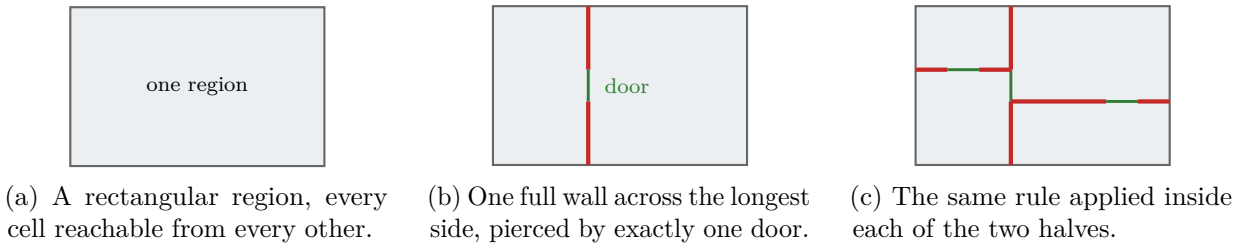


Figure 1: The construction rule. Each application turns one region into two regions of the same kind joined by a single door, which is what makes this a divide-and-conquer problem and the decomposition a binary tree.

What “at random” binds. Half the output, not all of it.

The cells are fixed by W and H . The same WH positions appear in every run, and their coordinates are public the moment the dimensions are.

What must be unpredictable is **which doors were kept** — the edge set.

So the requirement falls on the draws, not on the storage. A rule that takes the wall from the geometry, the midpoint say, makes the maze a function of W and H . Anyone holding those two numbers could rebuild it.

1.2 From a drawing of walls to a graph

A maze on paper is a picture of walls. What an algorithm can generate, store and search is a graph.

That translation is the first decision of this report, and every later section rests on it.

Definition 1 (Area, cells, grid graph). Let $W, H \in \mathbb{N}^*$ be the width and the height of the area, in unit cells. The cells are

$$V = \{(x, y) \in \mathbb{N}^2 : 0 \leq x < W, 0 \leq y < H\}, \quad |V| = WH,$$

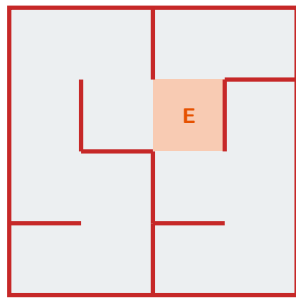
and two cells are *adjacent* when they share a side, that is when $|x - x'| + |y - y'| = 1$. The *grid graph* $G_{\text{grid}} = (V, E_{\text{grid}})$ has the cells as vertices and the adjacent pairs as edges, so

$$|E_{\text{grid}}| = \underbrace{H(W - 1)}_{\text{horizontal}} + \underbrace{W(H - 1)}_{\text{vertical}} = 2WH - W - H.$$

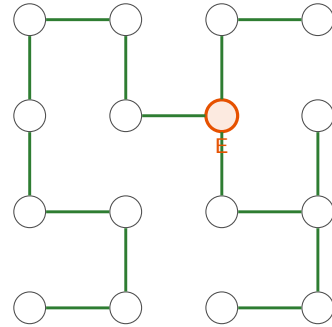
Definition 2 (Maze). A *maze* on that area is a spanning subgraph $M = (V, E)$ with $E \subseteq E_{\text{grid}}$. An edge of E is a **door**: the two cells it joins can be walked between. A pair of $E_{\text{grid}} \setminus E$ is a **wall segment**. One distinguished cell v_{exit} is the exit.

Figure 2 shows the two readings of one object.

The vertex set is fixed by W and H alone. So generating a maze means nothing more than **choosing which subset of E_{grid} to keep**.



(a) The maze as walls. The area is enclosed; the exit is a cell, not a gap in the outer wall.



(b) The same maze as a spanning tree of the grid graph.

Figure 2: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The exit is a vertex marked on the finished tree — here an interior one, with degree 3 — and neither its position nor its degree affects anything the construction does.

1.3 What the specification forces E to be

Three requirements follow from the statement.

- (R1) **Every cell reaches the exit.** The entry position is arbitrary, so any cell may be a start and each must have a route out. M is connected.
- (R2) **There is exactly one exit.** One cell is designated v_{exit} . It is a *cell*, not a gap in the outer wall: the area is enclosed on every side, and the exit is the one place leading out of it. It may lie anywhere, including strictly inside.
- (R3) **The route out does not fork.** The path to the exit is required to be unique, hence shortest. A cycle in M would give two distinct simple routes between the cells it passes through, so M is acyclic.

Connected and acyclic on a fixed vertex set is a spanning tree. The shape of the answer is settled before any algorithm is written.

Proposition 1 (The object to produce). *A subgraph $M = (V, E)$ of G_{grid} satisfies (R1) and (R3) if and only if it is a spanning tree of G_{grid} . It then has $|E| = WH - 1$ doors, and between any two cells there is exactly one simple path, which is therefore also a shortest one.*

That last clause is what makes Problem 2 well posed. Asking for *the* path out is only meaningful because there is one.

It is also the lever for the rest of the report: Section 2 produces such a tree **by construction**, rather than by checking afterwards.

1.4 Constraints on the exit

Three questions about v_{exit} , none of which constrains the construction.

Chosen before or after generation?

After. Generation takes W and H and nothing else, and no step of it could consult an exit. By Proposition 1 every cell is reachable from every other, so any cell designated afterwards works.

On the boundary?

No. The exit is a cell of the area, not a hole in the wall enclosing it, so nothing obliges it to touch the perimeter.

Exactly one door?

No. Uniqueness of the route out comes from M being a tree, not from the exit's degree: with 1, 2, 3 or 4 doors there is still exactly one simple path to it from anywhere.

The exit carries no constraint. It is a **label applied to a finished tree**, and every cell is an equally valid candidate. Figure 2 marks an interior one of degree 3.

1.5 The two problems, restated on the graph

Problem 1 — Generation.

Given W , H and a source of random bits, produce a spanning tree M of G_{grid} , drawn so that no two runs need agree, together with the structure recording how it was built. The exit is not an input.

Problem 2 — Escape.

Given M and any start cell $s \in V$, return the unique simple path from s to v_{exit} .

Problem 1 is solved in Section 2.

Problem 2 needs a notion of length first. Section 3 gives every door weight 1; Section 4 runs the shortest-path search that weight makes meaningful, guided by the Manhattan and Euclidean distances.

1.6 Conventions

Symbol	Meaning
W, H	width and height of the area, in cells
$n = WH$	number of cells, the size of the input
(x, y)	cell in column x (left to right), row y (top to bottom)
$G_{\text{grid}}, E_{\text{grid}}$	the grid graph and its edges: every adjacency
$M = (V, E)$	the maze produced: a spanning tree of G_{grid}
v_{exit}	the single exit cell, anywhere in V
$(x_0, y_0) - (x_1, y_1)$	a region, by its upper-left and lower-right cells

1.7 Scope of the deliverable

The specification fixes the contents, not only the topic: a data structure and what it holds, an algorithm per problem, the size of that structure, and the cost of both algorithms.

Required	Where	State
Data structure, and what it stores	Section 2	done
Maze generation algorithm	Section 2	done
Cost of a door, and of a path	Section 3	to write
Path-finding algorithm	Section 4	to write
Implementation (optional)	Section 5	to write
Size of the structure, cost of both algorithms	Section 6	to write

2 Generating the maze into a graph

Section 1 fixed the target: a spanning tree of G_{grid} , drawn at random.

This section builds one. The structure comes first, because the algorithm is written against it. The guarantee comes last, because the construction rule is what supplies it.

2.1 What has to be stored

Two different things, and confusing them is the usual way to store neither well.

The maze — which doors exist.

What a walker needs, and all that Problem 2 requires.

The way it was built — the region decomposition.

Which wall went where, in what order. The binary tree the divide-and-conquer rule produces for free, and the object Section 6 measures.

2.2 The node record

The maze is a set of nodes, one per cell. A node holds

$$\text{Node} = \langle \underbrace{\text{value}}_{\text{its } (x,y) \text{ coordinates}}, \underbrace{\text{neighbours}}_{\text{the cells it has a door to}} \rangle.$$

The neighbour list is **empty when the node is created**. A cell has four sides, so it never holds more than four entries, and it grows only when the construction puts a **door** on that node.

An index $\langle P_i, P_j \rangle$ — a $W \times H$ array of references — maps a coordinate pair to its node in $O(1)$. A division reaches the cells it wants without searching. Figure 3 is one node before and after.

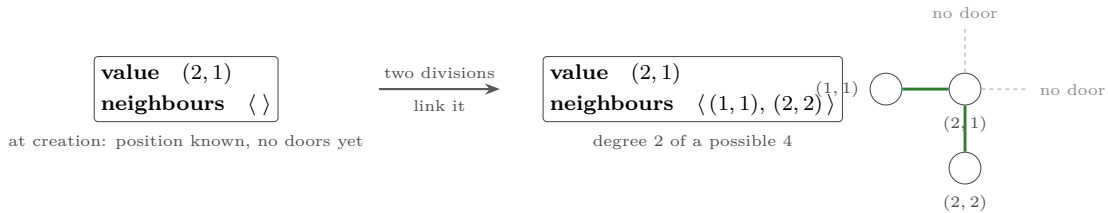


Figure 3: The node record. A node carries its coordinates and a list of the neighbours it has a door to; the list is empty when the node is made and is appended to only by a division whose door lands on that node. A cell has four sides, so the list never exceeds four entries, and in the finished maze the entries across all nodes number $2(WH - 1)$ — each door recorded once at each of its two ends.

Each door is recorded at both ends, so the finished maze holds $2(WH - 1)$ entries for $WH - 1$ doors. That buys enumeration of a cell's exits in $O(\deg)$ from the cell alone, which is what Section 4 does at every step.

2.3 Regions, named by two corners

A region is a rectangle of cells, named by the two cells at its ends: $(x_0, y_0) - (x_1, y_1)$, upper-left and lower-right, with $x_0 \leq x_1$ and $y_0 \leq y_1$. The whole area is $(0, 0) - (W - 1, H - 1)$.

Both corners are **cells of the region**, not fences around it. A region spans $x_1 - x_0 + 1$ columns and $y_1 - y_0 + 1$ rows.

In the pseudocode the two corners are p and q , and a cell's coordinates are reached by index: $p[0]$ is its x , $p[1]$ its y . Indexing rather than naming them lets the division below be written once for both axes.

2.4 The recursion stops at a corridor

Definition 3 (Corridor). A region is a *corridor* when one of its two sides is a single cell: $x_0 = x_1$, one column wide and any height, or $y_0 = y_1$, one row tall and any width. A single cell, where both hold, is the smallest corridor.

A corridor is the case where the construction has nothing to decide.

Its cells lie in a line, so the subgraph G_{grid} induces on it is already a path. A path on m vertices has $m - 1$ edges and is a tree, so it is its own and only spanning tree.

Put a **wall** across a corridor and it has exactly one crossing, which is then forced to be the **door**. The division would change nothing.

So the recursion stops there. **SpanCorridor** lays those forced doors and returns the leaf recording them.

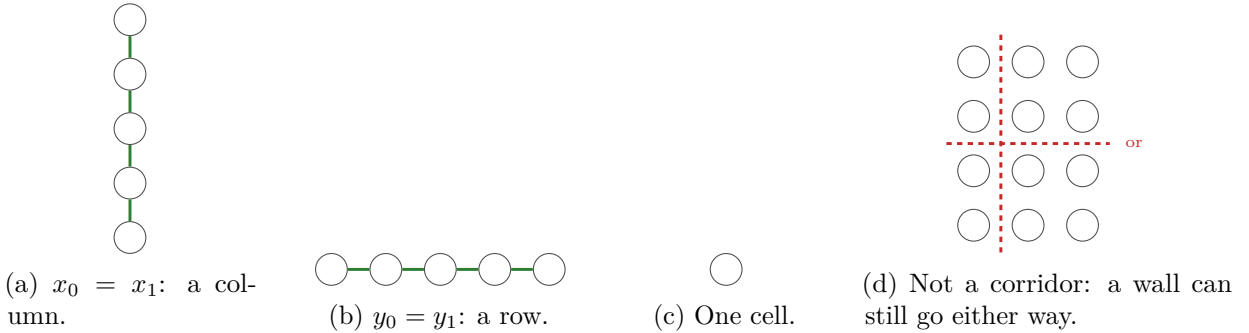


Figure 4: A *corridor* is a region one cell thick, in either direction. Its cells lie in a line, so the induced graph is already a path and there is exactly one way to span it — nothing about a corridor is left to decide, which is why the recursion stops at one. A region thicker than that, on the right, still has both a choice of orientation and a choice of position.

Stopping at a corridor means **every region reaching a division has both sides at least 2**. Both axes always admit a wall, so the orientation is a plain coin with no test in front of it.

2.5 One division contributes exactly one door

Take any door $e = (u, v)$ of the finished maze and walk down the region tree from the root.

At each internal node, u and v either fall in the same child — and the walk continues — or they are separated. They are separated exactly when e crosses that node's wall, and then e is that node's door, being the only crossing there is.

Otherwise the walk ends at a leaf holding both, and e is one of the doors that leaf laid along its corridor.

So the doors split cleanly in two: one per internal node, plus the interiors of the corridor leaves. Nothing is counted twice, nothing is missed.

That licenses the construction to run **forwards**. A division adds its own door and nothing else; a leaf adds its corridor. A wall is never represented at all — it is simply the doors that were never added.

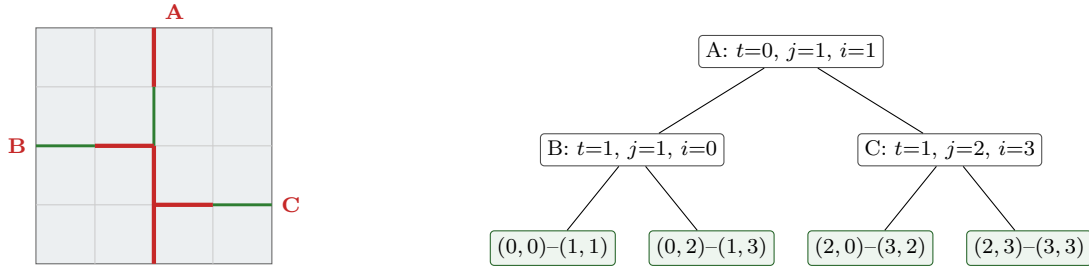
2.6 The region tree

The recursion's shape is a binary tree, one node per call.

- An **internal node** records one division: its orientation t , the boundary j the wall sits on, the position i of its door. Its two children are the two regions the division created.
- A **leaf** is a corridor, of one cell or many.

A binary tree with L leaves has $L - 1$ internal nodes. So of the $WH - 1$ doors, $L - 1$ are division doors and $WH - L$ are corridor doors.

Figure 5 shows the first three divisions of a 4×4 area beside the top of the tree they build.



(a) The first three divisions of a 4×4 area.

(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be divided, not corridors.

Figure 5: Every division is one internal node. The node records the orientation t , the wall position j and the door position i it drew, and its two children are the two regions it created; a leaf is a corridor. A tree with L leaves has $L - 1$ internal nodes, so of the $WH - 1$ doors of Proposition 1, $L - 1$ are division doors and the rest are laid down inside the corridors.

2.7 One division step

A call handles a region $(x_0, y_0) - (x_1, y_1)$. If it is a corridor, Section 2.4 disposes of it. Otherwise both sides are at least 2, and:

1. **Toss for the axis t to cut.** Both admit a wall, so the coin is free. Write $t = 0$ for the x axis and $t = 1$ for the y axis; then $1 - t$ is the other one.
2. **Draw the wall position j on axis t ,** uniformly in $[p[t], q[t] - 1]$, **and the door position i on the other axis,** uniformly in $[p[1 - t], q[1 - t]]$.
3. **Link the two cells the door falls on,** appending each to the other's neighbour list.

4. **Recurse** on the two regions — the same corners, cut at j on axis t — and hang them under the node recording $\langle t, j, i \rangle$.

Remark (Departure from the specification). The specification puts the wall across the *longest* dimension. This report tosses for it instead. Correctness is untouched: Proposition 2 never uses which axis was cut, only that one was.



(a) The call links one pair and stops. Every other node is still isolated; no crossing was examined, let alone removed.

(b) After both recursive calls return. Each half is a tree, and the door is still the only edge between them.

Figure 6: One division, with $t = 0$ — the coin named the x axis, so the wall is vertical. The call draws the wall position j on that axis and the door position i on the other, links the two cells the door falls on, and recurses on the two halves. Counting edges: 11 on the left, 7 on the right, one door, total $19 = 20 - 1$ on 20 cells — the tally of Proposition 2.

2.8 The algorithm

Algorithm 1: Maze generation by recursive division.

Input: $W, H \geq 1$, the width and height of the area in cells. The bound is a precondition, not a convention: at $W = 0$ the region $(0, 0) - (-1, H - 1)$ holds no cell and there is nothing for the first draw to land on.

Output: N , the set of nodes holding a spanning tree of G_{grid} ; r , the root of the region tree that built it.

```

1 Function GenerateMaze( $W, H$ ):
2    $N \leftarrow \{ \text{NewNode}(x, y) : 0 \leq x < W, 0 \leq y < H \}$ ;    // every neighbour list empty
3    $r \leftarrow \text{Divide}(N, (0, 0), (W - 1, H - 1))$ ;
4   return ( $N, r$ );

5 Function Divide( $N, p, q$ ):
6   if  $p[0] = q[0]$  or  $p[1] = q[1]$  then return SpanCorridor( $N, p, q$ );
7    $t \leftarrow \text{RandomBit}()$ ;                                // the axis to cut: 0 is  $x$ , 1 is  $y$ 
8    $j \leftarrow \text{RandomIn}(p[t], q[t] - 1)$ ;                // wall between  $j$  and  $j + 1$  on axis  $t$ 
9    $i \leftarrow \text{RandomIn}(p[1-t], q[1-t])$ ;                // the one door, at  $i$  on the other axis
10   $\text{Link}(N, \text{CellAt}(t, j, i), \text{CellAt}(t, j+1, i))$ ;
11   $\ell \leftarrow \text{Divide}(N, p, \text{WithAxis}(q, t, j))$ ;
12   $r \leftarrow \text{Divide}(N, \text{WithAxis}(p, t, j+1), q)$ ;
13  return CutNode( $t, j, i, \ell, r$ );

14 Function SpanCorridor( $N, p, q$ ):
15  for  $x \leftarrow p[0]$  to  $q[0] - 1$  do  $\text{Link}(N, (x, p[1]), (x+1, p[1]))$ ;
16  for  $y \leftarrow p[1]$  to  $q[1] - 1$  do  $\text{Link}(N, (p[0], y), (p[0], y+1))$ ;
17  return Leaf( $p, q$ );

18 Function Link( $N, u, v$ ):
19    $\text{append } N[v] \text{ to } N[u].\text{neighbours}$ ;
20    $\text{append } N[u] \text{ to } N[v].\text{neighbours}$ ;

21 Function CellAt( $t, u, v$ ):
22  return the cell whose axis- $t$  coordinate is  $u$  and whose other is  $v$ ;

23 Function WithAxis( $p, t, u$ ):
24  return the cell  $p$  with its axis- $t$  coordinate replaced by  $u$ ;
```

`SpanCorridor()` is the whole base case. It lays the corridor's forced doors *and* returns the leaf recording the region.

It needs no case analysis: on a corridor one of its loops is empty by construction, on a single cell both are. The same two loops cover a row, a column and one cell.

Every `Link()` joins two cells of the region that called it, and they are adjacent. So every appended entry is a genuine edge of G_{grid} .

The exit plays no part, for the reasons in Section 1.4. The algorithm never reads it, and any cell may be designated once the tree exists.

2.9 Why the result is a spanning tree

Proposition 2 (Correctness of Algorithm 1). *On a region of m cells, Divide leaves the subgraph induced on that region connected and acyclic, with exactly $m - 1$ doors. In particular GenerateMaze returns a spanning tree of G_{grid} .*

Proof. By induction on m .

Base. If the region has a side of one it is a corridor of m cells in a line, and **SpanCorridor** joins each to the next: a path, connected and acyclic, carrying $m - 1$ doors. This covers $m = 1$, where both loops are empty and $m - 1 = 0$.

Step. Otherwise both sides are at least 2, so $q[0] - p[0] \geq 1$ and $q[1] - p[1] \geq 1$. Whichever axis the coin names, $q[t] - p[t] \geq 1$, so j is drawn from a non-empty range and the two sub-regions R_ℓ and R_r are non-empty, with $m_\ell + m_r = m$.

The call adds the single door at i , whose two cells lie one in each sub-region, then recurses. By the induction hypothesis the two calls leave their regions connected and acyclic with $m_\ell - 1$ and $m_r - 1$ doors. Neither can add an edge between the sub-regions, since every **Link** a call makes joins two cells of its own region.

So the door at i is the only edge between R_ℓ and R_r . Joining two disjoint trees by a single edge yields a tree. It is connected, since any two cells are joined inside their own half or by crossing that door. It is acyclic, since a cycle would have to cross the wall an even, hence non-zero, number of times while only one crossing exists. The door count is $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$.

Applied to the whole area this gives a connected acyclic spanning subgraph of G_{grid} on all WH cells: a spanning tree, with $WH - 1$ doors. $\square$

The guarantee is structural. Nothing is checked after the fact and no attempt is ever rejected and retried.

The uniqueness of the route out — the whole of Problem 2 — follows from the shape of the recursion, not from a property that had to be tested.

2.10 The graph that comes out

Generation returns the set N of nodes, each carrying its coordinates and the doors it ended up with, plus the root r of the region tree.

Three readings of that one result, none of them a separate copy:

View	Held as	Used by
Adjacency	each node's neighbour list	the search of Section 4
Position	each node's <i>value</i>	the heuristics of Section 4
How it was built	the region tree rooted at r	the cost analysis of Section 6

The second row is what makes Section 4 possible. A heuristic estimating the remaining distance to the exit needs the coordinates of the cell it stands on, and they are in the node.

Figure 2b is the output of one 4×4 run, whose first three divisions are drawn in Figure 5a: 15 doors on 16 cells, one path between any two of them.

Section 3 adds a length to those doors, so that “the path out” becomes a quantity a search can minimise.

3 Encoding the distance between adjacent cells as 1

To be written.

4 Shortest path to the exit, with distance as heuristic guidance

To be written.

5 Testing with a Python implementation

To be written.

6 Complexity and analysis of the results

To be written.